Experiment No. – 9							
Date of Performance:	**3rd Aug 2026**						
Date of Submission:	**4th Aug 2026**						
Conceptual Understanding (02)	**Execution (1.5)**	**Problem Solving & Debugging(02)**	**Professional Conduct & Lab Ethics(02)**	**Output/ Result Accuracy (1.5)**	**Documentation & Result Analysis (01)**	**Experiment Total (10)**	**Sign with Date**

Experiment No. 9
Create a Banking application using Exception handling

9.1 Aim: Create a Banking application using Exception handling:Create banking application with various functionality like opening account, deposit funds and withdraw funds, create user defined exception for maintaining minimum balance while performing withdraw operation.

9.2 Course Outcome: Apply Multithreading and Exception handling concepts to develop efficient Java Program

9.3 Learning Objectives: To show how the program/software can run without any interruption even after encountering run time errors.

9.4 Requirement: Java development kit (JDK as per OS)

9.5 Related Theory:

The Exception Handling in Java is one of the powerful mechanism to handle the runtime errors to ensure the normal flow of the application can be maintained. Exception Handling is a mechanism to handle runtime errors such as ClassNotFoundException, IOException, SQLException,

RemoteException, etc.
An exception normally disrupts the normal flow of the application, which makes it crucial to handle the exceptions.
Types of Java Exceptions: There three types of exceptions namely:
Checked Exception : The classes that directly inherit the Throwable class except RuntimeException and Error are known as checked exceptions. For example, IOException, SQLException, etc. Checked exceptions are checked at compile-time.
Unchecked Exception : The classes that inherit the RuntimeException are known as unchecked exceptions. For example, ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, etc. Unchecked exceptions are not checked at compile-time, but they are checked at runtime.
Error: Error is irrecoverable. Some examples of errors are OutOfMemoryError, VirtualMachineError, AssertionError etc. In Java, an exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.

Catching Exceptions
A method catches an exception using a combination of the try and catch
keywords. A try/catch block is placed around the code that might generate an exception. Code within a try/catch block is referred to as protected code, and the syntax for using try/catch looks like the following –

Syntax:
```
try
{
// Protected code
 }
 catch (ExceptionName e1)
{
 //
Catch block }
```

Types of Exception in Java
1)Built-in exceptions
2)User-defined exception

1)Built-in exceptions Built-in exceptions are the exceptions that are available in Java library.

1- ArithmeticException: It is thrown when an exceptional condition has occurred in an arithmetic operation.
Example:
```
import java.io.*;
  class Arith
{
  public static void main (String[] args)
        {
                        int a=15;
                        int b=0;
              try
              {
                        System.out.println(a/b);
```

```
            }
              catch(ArithmeticException e)
            {
                    e.printStackTrace();
            }
        }
}
```

Output:

java.lang.ArithmeticException: / by zero

2-ArrayIndexOutOfBoundsException: It is thrown to indicate that an array has been accessed with an illegal index. The index is either negative or greater than or equal to the size of the array.

Example:

```
class ArrayIndex
{
public static void main(String args[])
        {
                try
                {
                        int a[] = new int[5];
                         a[6] = 9;
                 }
                 catch(ArrayIndexOutOfBoundsException e)
                {
                        System.out.println ("Array Index is Out Of Bounds");
                }
        }
}
```

Output:

Array Index is Out Of Bounds

3-ClassNotFoundException: This Exception is raised when we try to access a class whose definition is not found

Example:

```
public class ClassDemo
{
 public static void main(String[] args)
        {
                try
                {
                        Class.forName("Class1");
                 }
```

```
                catch(ClassNotFoundException e)
                {
                System.out.println(e);
                System.out.println("Class Not Found...");
            }
        }
}
```

Output:

java.lang.ClassNotFoundException: Class1

Class Not Found...

4-FileNotFoundException: This Exception is raised when a file is not accessible or does not open.
Example:

```
import java.io.File;
import java.io.FileNotFoundException;
import java.io.FileReader;
 class File_notFound_Demo
{

 public static void main(String args[])
        {
            try
            {
                    File file = new File("E://file.txt");
                    FileReader fr = new FileReader(file);
            } catch (FileNotFoundException e)
            {
                      System.out.println("File does not exist");
            }
        }
}
```

Output:

File does not exist

5-IOException: It is thrown when an input-output operation failed or interrupted
Example:

```
class IOException
 {
public static void main(String[] args)
          {
```

```
                Scanner scan = new Scanner("Java Programming");
                System.out.println("" + scan.nextLine());
                System.out.println("Exception Output: " + scan.ioException());
               scan.close();
        }
}
```

Output:

Java Programming
Exception Output: null

6-NoSuchFieldException: It is thrown when a class does not contain the field (or variable) specified
Example:

```
public class example
{
 public static void main(String[] args)
        {
                Set example Set = new HashSet();
                Hashtable exampleTable = new Hashtable();
                exampleleSet.iterator().next();
                exampleTable.elements().nextElement();
         }
}
```

7-NullPointerException: This exception is raised when referring to the members of a null object. Null represents nothing
Example:

```
class NullPr
{
 public static void main(String args[])
        {
                try
               {
                        String a = null;
                        System.out.println(a.charAt(0));
                 } catch(NullPointerException e)
                 {
                          System.out.println("NullPointerException..");
                }
        }
}
```

8-NumberFormatException: This exception is raised when a method could not convert a string into

a numeric format.

Example:

```
class  NumFormat
{
public static void main(String args[])
        {
                try
                {
                        int num = Integer.parseInt ("java") ;
                        System.out.println(num);
                } catch(NumberFormatException e)
                {
                        System.out.println("Number format exception");
                }
          }
}
```

Runtime Exception: This represents an exception that occurs during runtime.

9-StringIndexOutOfBoundsException: It is thrown by String class methods to indicate that an index is either negative or greater than the size of the string

Example:

```
class Strbound
{
public static void main(String args[])
  {
        try
        {
                String a = "welcome to sakec ";
                char c = a.charAt(24);
                System.out.println(c);
         }
         catch(StringIndexOutOfBoundsException e)
        {
                System.out.println("String Index Out Of Bounds Exception");
         }
   }
}
```

10-IllegalArgumentException : This exception will throw the error or error statement when the method receives an argument which is not accurately fit to the given relation or condition. It comes under the unchecked exception.

Example:

```
import java.io.*;
```

```
class Argu
{
public static void print(int a)
    {
                if(a>=18)
                {
                        System.out.println("Eligible for Voting");
                 }
                        else
                        {
                                throw new IllegalArgumentException("Not Eligible for Voting");
                         }
  public static void main(String[] args)
                {
                  Argu.print(14);
                }
        }
  }
```

11-IllegalStateException : This exception will throw an error or error message when the method is not accessed for the particular operation in the application. It comes under the unchecked exception.

Example:

```
import java.io.*;

class state
{
             public static void  print(int a,int b)
             {
                  System.out.println("Addition of Positive Integers :"+(a+b));
             }
 public static void main(String[] args)
             {
                  int n1=7;
                  int n2=-3;
                  if(n1>=0 && n2>=0)
                  {
                          state.print(n1,n2);
                    }
                  else
                   {
                          throw new IllegalStateException("Either one or two numbers are not
```

```
        Positive Integer");
                }
            }
}
```

12-InterruptedException: It is thrown when a thread is waiting, sleeping, or doing some processing, and it is interrupted.

13-NoSuchMethodException: It is thrown when accessing a method that is not found.

2) User-defined Exceptions

Users can also create exceptions which are called 'user-defined Exceptions'. The following steps are followed for the creation of a user-defined Exception. The user should create an exception class as a subclass of the Exception class.

Example:

class MyException extends Exception

We can write a default constructor in his own exception class.

MyException(){}

We can also create a parameterized constructor with a string as a parameter.

We can use this to store exception details. We can call the superclass(Exception) constructor from this and send the string there.

```
MyException(String str)
{
  super(str);
}
```

To raise an exception of a user-defined type, we need to create an object to his exception class and throw it using the throw clause, as:

MyException me = new MyException("Exception details");

throw me;

```
class MyException extends Exception
{
        public MyException(String s)
        {
                                super(s);
        }
}

public class Main
      {
public static void main(String args[])
        {
                try
                {
                        throw new MyException("Java Programming");
                }
                catch (MyException ex)
                {
                        System.out.println("Welcome");
```

```
                System.out.println(ex.getMessage());
            }
            }
    }
```

Java provides specific keywords for exception handling purposes.

throw: We know that if an error occurs, an exception object is getting created and then Java runtime starts processing to handle them. Sometimes we might want to generate exceptions explicitly in our code. For example, in a user authentication program, we should throw exceptions to clients if the password is null. The throw keyword is used to throw exceptions to the runtime to handle it.

throws: When we are throwing an exception in a method and not handling it, then we have to use the throws keyword in the method signature to let the caller program know the exceptions that might be thrown by the method. The caller method might handle these exceptions or propagate them to its caller method using the throws keyword. We can provide multiple exceptions in the throws clause, and it can be used with the main() method also.

try-catch – We use the try-catch block for exception handling in our code. try is the start of the block and catch is at the end of the try block to handle the exceptions. We can have multiple catch blocks with a try block. The try-catch block can be nested too. The catch block requires a parameter that should be of type Exception.

finally: the finally block is optional and can be used only with a try-catch block. Since exceptions halt the process of execution, we might have some resources open that will not get closed, so we can use the finally block. The finally block always gets executed, whether an exception occurred or not. We can't have a catch or finally clause without a try statement.

A try statement should have either catch block or finally block, it can have both blocks.

We can't write any code between try-catch-finally blocks. We can have multiple catch blocks with a single try statement. try-catch blocks can be nested similar to if-else statements. We can have only one finally block with a try-catch statement.

```
import java.io.FileNotFoundException;
import java.io.IOException;
public class ExceptionHandling
{

public static void main(String[] args) throws FileNotFoundException, IOException
        {
                try
                {
                        testException(-5);
                        testException(-10);
                } catch(FileNotFoundException e)
                {
                        e.printStackTrace();
                } catch(IOException e)
                {
                        e.printStackTrace();
                } finally
                {
                        System.out.println("Releasing resources");
                }
                testException(15);
```

```
                }

public static void testException(int i) throws FileNotFoundException, IOException
                {
                if (i < 0)
                {
                        FileNotFoundException myException = new
                FileNotFoundException("Negative Integer " + i);
                        throw myException;
                } else if (i > 10)
                {
                        throw new IOException("Only supported for index 0 to 10");
                }
        }
}
```

9.6 Procedure:

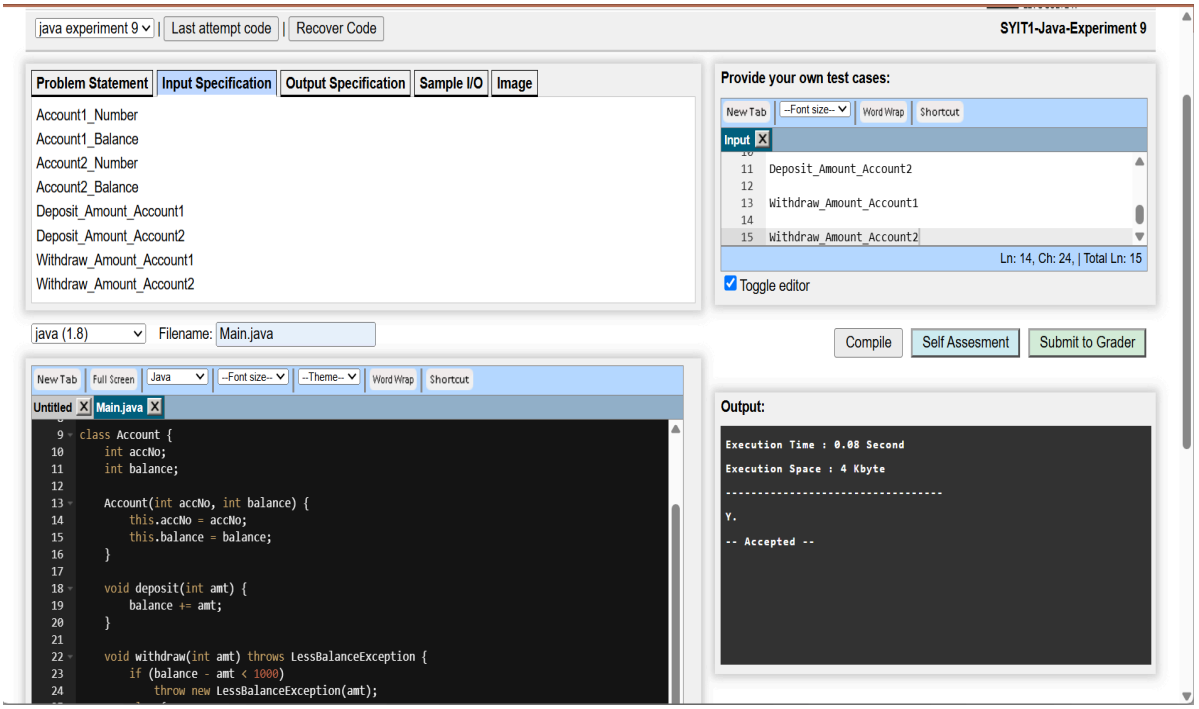
Create a java class which has various methods to perform banking tasks like, creating a account, deposit amount and withdraw amount. The withdraw method must throw an exception when the balance goes below required minimum balance. Create a user defined exception for handling minimum balance exception.

Form main class perform various operations by calling the respective methods.

9.7 Program and Output:

```
// User-defined exception for minimum balance
class MinimumBalanceException extends Exception {
    public MinimumBalanceException(String message) {
        super(message);
    }
}

// Banking class
class BankAccount {
    private String accountHolder;
    private double balance;
    private static final double MIN_BALANCE = 1000;

    public BankAccount(String accountHolder, double initialDeposit) {
        this.accountHolder = accountHolder;
        this.balance = initialDeposit;
    }

    public void deposit(double amount) {
        balance += amount;
        System.out.println("Deposited: " + amount + " | Current Balance: " + balance);
    }

    public void withdraw(double amount) throws MinimumBalanceException {
        if (balance - amount < MIN_BALANCE) {
            throw new MinimumBalanceException("Withdrawal denied! Minimum balance of
" + MIN_BALANCE + " must be maintained.");
        }
```

```
        balance -= amount;
        System.out.println("Withdrawn: " + amount + " | Current Balance: " + balance);
    }

    public void displayBalance() {
        System.out.println("Account Holder: " + accountHolder + " | Balance: " + balance);
    }
}

// Main class
public class BankingApp {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount("Shravani", 5000);

        acc.displayBalance();
        acc.deposit(2000);

        try {
            acc.withdraw(5500); // This will throw exception
        } catch (MinimumBalanceException e) {
            System.out.println("Exception: " + e.getMessage());
        }

        acc.displayBalance();
    }
}
```



9.8 Conclusion:

This experiment demonstrates how exception handling ensures smooth execution of a banking application even when runtime errors occur. By creating a user-defined exception (MinimumBalanceException), we enforce business rules like maintaining a minimum balance. The program continues running without interruption, showing the importance of handling exceptions gracefully.

9.9 Questions:

1. **What are the different types of error and explain in short?**

- **Compile-time errors**: Detected by the compiler before execution (e.g., syntax errors, missing semicolons).
- **Runtime errors (Exceptions)**: Occur during program execution (e.g., divide by zero, null pointer).
- **Logical errors**: Program runs but produces incorrect output due to flawed logic.

2. What is the impact if exceptions are not handled properly?

- Program may **terminate abruptly**, losing unsaved data.

- User experience becomes poor due to **unexpected crashes**.
- System resources may remain **unreleased** (files open, memory leaks).
- Application becomes **unreliable and unsafe**.

3. What is the working of `for`, `while`, and `do-while` loops?

for loop: Used when the number of iterations is known.
java
for(int i=0; i<5; i++) { System.out.println(i); }

-

while loop: Executes repeatedly while the condition is true.
java
int i=0; while(i<5) { System.out.println(i); i++; }

-

do-while loop: Executes at least once, then checks condition.
java
int i=0; do { System.out.println(i); i++; } while(i<5);

-

4. How to take input from the user?

Using **Scanner class** in Java:

```
java
import java.util.Scanner;
public class InputDemo {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter your name: ");
      String name = sc.nextLine();
      System.out.println("Hello, " + name);
      sc.close();
   }
```

}

Would you like me to also design a **flowchart or UML diagram** for this banking application? It would make your experiment report more visually appealing and easier to understand.